Onyx — Changes applied to Circle

Onyx · multi-process OS on Raspberry Pi 4

Onyx uses **Circle** (rsta2/circle) as its hardware-abstraction + driver layer, kept as close to upstream as possible. The handful of changes we need live in a **fork**, pulled in as a **git submodule**:

- **Submodule:** `circle/` → `https://github.com/stephaneweg/circle.git` (`.gitmodules`), branch **`onyx`**.
- **Upstream base:** `https://github.com/rsta2/circle.git`, tag **`Step51.1.1`** (remote `upstream`) — `Step51` until 2026-10-09, when upstream's 51.1 and 51.1.1 were merged in (see *Upstream merges* below).
- The superproject pins a specific fork commit; `git submodule status` shows it (currently `Step51-6-g…`).

The patches on top of upstream are small and surgical. The diffs below are the output of `git diff Step51..onyx` inside `circle/` at the time each was written (since the merge of 51.1.1, compare with `Step51.1.1`). To reproduce:

```
git -C circle remote -v                  # origin = the fork, upstream = rsta2/circle
git -C circle log  --oneline --no-merges Step51.1.1..onyx
git -C circle diff Step51.1.1..onyx
```

> **Design rule.** Patch Circle as little as possible. When a change can be header-only (no library rebuild), it is. Anything that can live in our own kernel instead (for example the cooked-mouse changed-button handling, done in `kernel/`'s mouse stub) is **not** a Circle patch.
>
> Some in-source comments still say "Zircon" (the project's former codename) instead of "Onyx" — cosmetic only.

Summary

#	Patch	Files touched	Rebuild
1	Keyboard map **decoupled** from the kernel (no compiled-in country maps; loaded at runtime) · `MAX_TASKS` 20 → 40	`input/keymap.{h,cpp}` (+ 7 `keymap_*.h` deleted), `usb/usbkeyboard.h`, `input/keyboardbehaviour.h`, `sysconfig.h`	`libinput` (+ kernel for `MAX_TASKS`)
2	Heap **large-block reuse** (fix per-launch canvas leak) + free-list byte accounting	`heapallocator.{h,cpp}`, `sysconfig.h`	`libcircle`

#	Patch	Files touched	Rebuild
3	**Free-space accounting** in the page allocator + memory accessors (for the `meminfo` kapi)	`memory.h`, `pageallocator.{h,cpp}`	`libcircle` (`memory.h` header-only)
4	**Multi-core enabled** (`ARM_ALLOW_MULTI_CORE`): core 1 runs the sound producer	`sysconfig.h`	**every library** (clean rebuild) + kernel
5	**Shift + navigation keys** (`KeyShiftUp`… appended to `TSpecialKey`, xterm `;2` sequences)	`input/keymap.{h,cpp}`	`libinput`, `libusb` + the `.kmap` files
6	**Partial display update** `C2DGraphics::UpdateDisplay (x, y, w, h)` (the compositor's dirty rectangles)	`2dgraphics.{h,cpp}`	`libcircle`
7	**Yielding SD waits**: weak hooks for the FatFs volume lock and the EMMC wait loop	`addon/fatfs/ffsystem.cpp`, `addon/SDCard/emmc.cpp`	`libfatfs`, `libsdcard`
8	**Sector cache** in the FatFs disk layer (write-through), one bounce buffer per volume	`addon/fatfs/diskio.{h,cpp}`	`libfatfs`
10	**Multi-cluster transfers** in `f_read` / `f_write` (one SD command across contiguous clusters)	`addon/fatfs/ff.c`	`libfatfs`
9	**High Speed SD at run time** (`CEMMCDevice::S`	`addon/SDCard/emmc.{h,cpp}`	`libsdcard`

#	Patch	Files touched	Rebuild
	`etHighSpeed`) + the host's High Speed Enable bit		
11	**FatFs fast seek** on (`FF_USE_FASTSEEK 1`: `kapi_seek` in a GameCube disc image)	`addon/fatfs/ffconf.h`	`libfatfs` + the kernel (the `FIL` layout: clean rebuild)
14	**DHCP restart** (joining another Wi-Fi network while running; the reconfiguration itself: the kernel's link-time wrap, no hostap change)	`include/circle/net/dhcpclient.h`, `lib/net/dhcpclient.cpp`	`libnet` (+ the kernel)
13	**The SD card's partitions as volumes** `SD:` `SD1:` … `SD3:` + **exFAT** on	`addon/fatfs/ffconf.h`, `addon/fatfs/diskio.cpp`, `addon/fatfs/ff.c`	`libfatfs` + the kernel (`FSIZE_t` is 64-bit: clean rebuild)
17	**The host name set after the constructor** (`CNetSubSystem::SetHostname`: `system.ini`'s `hostname=`)	`include/circle/net/netsubsystem.h`	none (inline) — the kernel
7b	**Yielding SD waits, many short ones**: a second weak hook `OnyxDriverPoll` at each turn of a short wait	`addon/SDCard/emmc.cpp`	`libsdcard`
20	**TCP: real duplicate ACKs only** (RFC 5681 §2) — the peer's data segments no longer start a fast retransmit	`lib/net/tcpconnection.cpp`	`libnet`
21	**TCP: RTO 1 s initial** (was 3 s;	`lib/net/retranstimeoutcalc.cpp`,	`libnet`

#	Patch	Files touched	Rebuild
	the 1 s minimum kept — 200 ms was tried and undone), Karn after a fast retransmit	`include/circle/net/retranstimeoutcalc.h`, `lib/net/tcpconnection.cpp`	
22	**`CSocket::Send`'s count**: the bytes queued when a later chunk times out	`lib/net/socket.cpp`	`libnet`
23	**Wi-Fi: the firmware's `wsec` from the IE's ciphers** (group + pairwise: TKIP 2, AES 4, as Linux's brcmfmac) -- WPA2 was always "aes": a WPA/WPA2 mixed-mode network (pairwise CCMP, group TKIP, common on 2.4 GHz) associated but no broadcast was decoded (no DHCP offer: the link stayed down)	`addon/wlan/ether4330.c` (`iewsec`, `setauth`)	`libwlan`
25	**TCP: a closed connection's retransmission timer** stopped, a late one ignored — it was a kernel panic (`Unexpected state 0`); a reset wakes a waiting sender	`lib/net/tcpconnection.cpp`	`libnet`
26	**Wi-Fi: the chip polled** when the network has its own core (the SDIO card interrupt came 2.5–5 ms late), **a scan over both**	`addon/wlan/ether4330.c`, `addon/wlan/emmc.c`, `addon/wlan/p9util.cpp`	`libwlan`

#	Patch	Files touched	Rebuild
	bands with the configured networks probed by name, **5 GHz preferred**, **a frame in one SDIO command**, the frames' locks without a yield, **the SDIO bus at 50 MHz**, **no A-MPDU for the frames sent** (half of them were lost during a download)		
27	**TCP: window scaling** (RFC 7323), **a receive window that follows the receive queue** (it was a constant), a 256 KB transmit threshold, **delayed acknowledgements** (one for eight segments)	`lib/net/tcpconnection.cpp`, `include/circle/net/tcpconnection.h`, `include/circle/net/sizes.h`	`libnet`
28	**USB volumes**: `f_mkfs` and the labels on (`FF_USE_MKFS`, `FF_USE_LABEL`), FatFs' objects checked again once the volume lock is held, the volume mutex kept across an unmount, a yield between two USB transfers, SCSI SYNCHRONIZE CACHE for the eject	`addon/fatfs/ffconf.h`, `ff.c`, `ffsystem.cpp`, `diskio.cpp`, `lib/usb/usbmassdevice.cpp`, `include/circle/usb/usbmassdevice.h`	`libfatfs`, `libusb` + the kernel (no structure changes: no clean rebuild)
29	**The Pi 5's high RAM registered once**: seg0 = [1	`lib/memory64.cpp`	`libcircle` (the Pi 5's, `circle5/`)

#	Patch	Files touched	Rebuild
	GB, 8 GB) is all of it; no reclaim above it (its fallback registered [4 GB, 8 GB) a second time: two allocators, the same frames); the crash record at seg0's top. The Pi 4 unchanged		
12	**2D DMA with a source stride** (a rectangle read in place: no gathering) + an **asynchronous** partial update (the compositor yields instead of spinning)	`dmachannel.{h,cpp}`, `dma4channel.{h,cpp}`, `bcmframebuffer.{h,cpp}`, `2dgraphics.{h,cpp}`	`libcircle`

1. Keyboard map decoupled from the kernel (+ `MAX_TASKS`)

Why. Circle's cooked-mode keyboard bakes the country maps into the binary (one selected at boot via `keymap=`) and offers no way to change it afterwards. Onyx wants **live** layout switching *and* layouts that ship as **data files** (`SD:/etc/keymaps/*.kmap`), so a new layout needs no kernel rebuild — and the kernel carries **no** keyboard tables at all.

What.

1. **Expose the keyboard's `CKeyMap`** up the device stack (`keyboardbehaviour.h`, `usbkeyboard.h` gain a `GetKeyMap()`), so the kernel can reach the live map and fill it through the existing public `CKeyMap::SetEntry` / `ClearTable`.
2. **Drop the compiled-in maps.** `keymap.cpp` no longer `#include`s the `keymap_*.h` tables, and `CKeyMap` loses `s_DefaultMap[]`, `s_MapDirectory[]` and `LookupDefaultMap()`; the constructor just zeroes the map (every entry `KeyNone`). The 7 stock tables (`keymap_{de,dv,es,fr,it,uk,us}.h`, **905 lines**) are **deleted** from the Circle tree — they now live in the Onyx repo at `tools/keymaps/maps/` and are compiled to `.kmap` blobs by `tools/keymaps/genkeymaps.py`.

The kernel fills the live keyboard from a `.kmap` file via `SetEntry` (`kapi_set_keymap_data`, ABI v27), re-applying it whenever a keyboard (re-)attaches. `MAX_TASKS` goes **20 → 40**: the network stack adds long-lived background tasks (net / DHCP / WPA supplicant + NTP + IRC). (Onyx's own scheduler has since dropped the fixed table: its tasks are a linked list with no limit, docs/02 §5, so `MAX_TASKS` no longer bounds Onyx; the patch stays for Circle's own code.)

```
--- a/include/circle/input/keymap.h
+++ b/include/circle/input/keymap.h
```

```
@@ -130,8 +130,9 @@ public:

    u8 GetLEDStatus (void) const;

-private:
-   static const void *LookupDefaultMap (const char *pLocale);
+   // Zircon: the keyboard map is empty until the kernel fills it at runtime via
+   // ClearTable/SetEntry from a SD:/etc/keymaps/*.kmap file -- no country map is
+   // compiled in (see CKeyMap::CKeyMap and kapi_set_keymap_data).

 private:
    u16 m_KeyMap[PHY_MAX_CODE+1][K_CTRLTAB+1];
@@ -141,8 +142,6 @@ private:
    boolean m_bScrollLock;

    static const char *s_KeyStrings[KeyMaxCode-KeySpace];
-   static const u16 s_DefaultMap[][PHY_MAX_CODE+1][K_CTRLTAB+1];
-   static const char *s_MapDirectory[];
 };

--- a/lib/input/keymap.cpp
+++ b/lib/input/keymap.cpp
@@ -96,60 +96,16 @@ const char *CKeyMap::s_KeyStrings[KeyMaxCode-KeySpace] =
    "."         // KeyKP_Period
 };

-#define C(chr)     ((u16) (u8) (chr))
-
-const u16 CKeyMap::s_DefaultMap[][PHY_MAX_CODE+1][K_CTRLTAB+1] =
-{
-   {
-       #include "keymap_de.h"
-   }, {
-       #include "keymap_dv.h"
-   }, {
-       ... (fr, es, it, uk, us)
-   }
-};
-
-const char *CKeyMap::s_MapDirectory[] = { "DE","DV","ES","FR","IT","UK","US", 0 };
-
+// Zircon: the kernel compiles in NO country maps -- keyboard layouts ship as
+// SD:/etc/keymaps/*.kmap data files and are loaded at runtime through
ClearTable/SetEntry
+// (see kapi_set_keymap_data). So a fresh keymap starts empty (every entry KeyNone).
 CKeyMap::CKeyMap (void)
 :  m_bCapsLock (FALSE), m_bNumLock (FALSE), m_bScrollLock (FALSE)
 {
-   const char *pLocale = CKernelOptions::Get ()->GetKeyMap ();
-   const void *pDefaultMap = LookupDefaultMap (pLocale);
-   ... (fallback to DEFAULT_KEYMAP)
-   memcpy (m_KeyMap, pDefaultMap, sizeof m_KeyMap);
+   memset (m_KeyMap, 0, sizeof m_KeyMap);      // KeyNone == 0
 }
@@ -336,17 +292,3 @@ u8 CKeyMap::GetLEDStatus (void) const
-const void *CKeyMap::LookupDefaultMap (const char *pLocale)
-{
-   for (unsigned nMap = 0; s_MapDirectory[nMap] != 0; nMap++)
-       if (strcmp (s_MapDirectory[nMap], pLocale) == 0)
-           return s_DefaultMap[nMap];
-   return 0;
```

```
-}

--- a/include/circle/input/keyboardbehaviour.h
+++ b/include/circle/input/keyboardbehaviour.h
@@ -69,6 +69,8 @@ public:
    u8 GetLEDStatus (void) const;

+   CKeyMap *GetKeyMap (void)   { return &m_KeyMap; }   // Zircon: runtime layout
switch
+
 private:
    void GenerateKeyEvent (u8 ucKeyCode);

--- a/include/circle/usb/usbkeyboard.h
+++ b/include/circle/usb/usbkeyboard.h
@@ -62,6 +62,9 @@ public:
    boolean SetLEDs (u8 ucStatus);

+   // Zircon: access the cooked-mode key map so the layout can be switched at
runtime.
+   CKeyMap *GetKeyMap (void)   { return m_Behaviour.GetKeyMap (); }
+
 private:
    void ReportHandler (const u8 *pReport, unsigned nReportSize);

--- a/include/circle/sysconfig.h
+++ b/include/circle/sysconfig.h
@@ -273,7 +277,8 @@
 #ifndef MAX_TASKS
-#define MAX_TASKS       20
+#define MAX_TASKS       40      // Onyx: bumped from 20 -- net stack adds
+                        // background tasks (net/DHCP/WPA + NTP/IRC)
 #endif
```

(The 7 `lib/input/keymap_*.h` tables are deleted — 905 lines — and not shown here.)

2. Heap large-block reuse (fix per-launch leak)

Why. Circle's `CHeapAllocator` keeps freed blocks on per-size **bucket** free lists, but the buckets top out at `0x80000` = 512 KB. A block **bigger than the top bucket cannot be returned to any free list and is lost** on `Free()`. Onyx allocates large blocks routinely — a window **canvas** is up to ~3 MB (1024×768×4) — so **every app launch leaked its canvas**.

Fix. Add **power-of-two "large" free lists** alongside the buckets: a too-big block is rounded to its power-of-two class and pushed onto `m_pLargeFreeList[exp]` on free, and reused from it on allocate. A running `m_nFreeListBytes` counter (exposed via `GetFreeListSpace()`) feeds the `meminfo` accounting (patch 3). **Rebuild `libcircle`** after this change.

```
--- a/include/circle/heapallocator.h
+++ b/include/circle/heapallocator.h
@@ -40,6 +40,12 @@ ASSERT_STATIC (DATA_CACHE_LINE_LENGTH_MAX >= 16);
 #define HEAP_BLOCK_MAX_BUCKETS 20

+// Onyx: blocks larger than the biggest bucket are rounded up to a power of two and
+// kept on a per-power free list (index = log2 of the rounded size), so large blocks
+// (window canvases, big file buffers, ...) are reused instead of lost on free. 32
+// classes cover any realistic size.
+#define HEAP_LARGE_LISTS   32
+
```

```
 struct THeapBlockHeader
@@ -83,6 +89,10 @@ public:
     size_t GetFreeSpace (void) const;

+    /// \return Onyx: bytes of freed blocks currently on the bucket/large free lists
+    ///     (reusable). Add to GetFreeSpace() for the true total free space.
+    size_t GetFreeListSpace (void) const    { return m_nFreeListBytes; }
+
     /// \param nSize Block size to be allocated
@@ -95,8 +105,8 @@ public:
     /// \param pBlock Memory block to be freed
-    /// \note Memory space of blocks, which are bigger than the largest bucket size,
-    ///   cannot be returned to a free list and is lost.
+    /// \note Onyx: blocks bigger than the largest bucket are rounded to a power of two
+    ///   and returned to a per-power free list (reused), so they are NOT lost.
     void Free (void *pBlock);
@@ -117,6 +127,8 @@ private:
     THeapBlockBucket m_Bucket[HEAP_BLOCK_MAX_BUCKETS+1];
+    THeapBlockHeader *m_pLargeFreeList[HEAP_LARGE_LISTS];   // per-power-of-2 (Onyx)
+    size_t       m_nFreeListBytes;  // Onyx: bytes on the bucket+large free lists
     CSpinLock    m_SpinLock;

--- a/lib/heapallocator.cpp
+++ b/lib/heapallocator.cpp
@@ -53,6 +53,45 @@ void CHeapAllocator::Setup (uintptr nBase, size_t nSize, size_t nReserve)
     m_nReserve = nReserve;
+
+    for (unsigned i = 0; i < HEAP_LARGE_LISTS; i++)     // Onyx: large-block free lists
+        m_pLargeFreeList[i] = 0;
+    m_nFreeListBytes = 0;
+}
+
+// Onyx: round a too-big request up to a power-of-two class; return its free-list index.
+static int HeapLargeClass (size_t *pnSize)
+{
+    size_t nRounded = HEAP_BLOCK_ALIGN; int nExp = 0;
+    while (nRounded < *pnSize) { nRounded <<= 1; if (++nExp >= HEAP_LARGE_LISTS) return -1; }
+    *pnSize = nRounded; return nExp;
+}
+// Free-list index of an already-rounded large block (stored size = ALIGN << exp).
+static int HeapLargeIndex (size_t nRoundedSize)
+{
+    int nExp = 0;
+    while (((size_t) HEAP_BLOCK_ALIGN << nExp) < nRoundedSize)
+        if (++nExp >= HEAP_LARGE_LISTS) return -1;
+    return nExp;
 }
@@ -96,12 +135,28 @@ void *CHeapAllocator::DoAllocate (size_t nSize)
+    // Onyx: too big for every bucket -> round to a power-of-two class so it can be
+    // reused from m_pLargeFreeList instead of being lost on free.
+    int nLargeExp = -1;
+    if (pBucket->nSize == 0) nLargeExp = HeapLargeClass (&nSize);
+
     THeapBlockHeader *pBlockHeader;
     if (pBucket->nSize > 0 && (pBlockHeader = pBucket->pFreeList) != 0) {
         pBucket->pFreeList = pBlockHeader->pNext;
```

```
+       m_nFreeListBytes -= pBlockHeader->nSize;    // Onyx
+   } else if (nLargeExp >= 0 && (pBlockHeader = m_pLargeFreeList[nLargeExp]) != 0) {
+       m_pLargeFreeList[nLargeExp] = pBlockHeader->pNext;
+       m_nFreeListBytes -= pBlockHeader->nSize;    // Onyx
    } else { ... bump m_pNext ... }
@@ -226,6 +281,7 @@ void CHeapAllocator::DoFree (void *pBlock)   // (bucket free
path)
            pBucket->pFreeList = pBlockHeader;
+           m_nFreeListBytes += pBlockHeader->nSize;    // Onyx
@@ -237,6 +293,19 @@ void CHeapAllocator::DoFree (void *pBlock)
+   // Onyx: no matching bucket -> power-of-two "large" block: return it to its size
+   // class free list so it is reused on the next same-class allocation (no leak).
+   int nLargeExp = HeapLargeIndex (pBlockHeader->nSize);
+   if (nLargeExp >= 0) {
+       m_SpinLock.Acquire ();
+       pBlockHeader->pNext = m_pLargeFreeList[nLargeExp];
+       m_pLargeFreeList[nLargeExp] = pBlockHeader;
+       m_nFreeListBytes += pBlockHeader->nSize;
+       m_SpinLock.Release ();
+       return;
+   }
```

```
--- a/include/circle/sysconfig.h
+++ b/include/circle/sysconfig.h
@@ -85,6 +85,10 @@
 #ifndef HEAP_BLOCK_BUCKET_SIZES
+// Small fixed buckets for common sizes. Larger requests (e.g. window canvases) no
+// longer need a matching bucket: Onyx's heap allocator rounds anything above the
top
+// bucket up to a power-of-two class and reuses it from a per-power free list (see
+// HEAP_LARGE_LISTS / heapallocator.cpp), so they are no longer lost on free.
 #define HEAP_BLOCK_BUCKET_SIZES    0x40,0x400,0x1000,0x4000,0x10000,0x40000,0x80000
 #endif
```

3. Free-space accounting (for the `meminfo` kapi)

Why. The `meminfo` kapi (and the memory monitor) reports total / free / app memory. The "free" figure must include both the unallocated region *and* the freed blocks/pages sitting on the allocators' free lists (reusable). Circle exposed only the unallocated region, so we add free-list accessors. `memory.h` is header-only; the `pageallocator` counter needs a `libcircle` rebuild.

```
--- a/include/circle/memory.h
+++ b/include/circle/memory.h
@@ -155,6 +155,25 @@ public:
    static void PageFree (void *pPage)  { s_pThis->m_Pager.Free (pPage); }

+   // Onyx: free space (bytes) of the page allocator region not yet handed out.
+   static size_t GetPagerFreeSpace (void)  { return s_pThis->m_Pager.GetFreeSpace
(); }
+   // Onyx: + freed pages on the pager free list (reusable).
+   static size_t GetPagerFreeListSpace (void) { return s_pThis-
>m_Pager.GetFreeListSpace (); }
+
+   // Onyx: freed heap blocks on the bucket/large free lists (reusable). Add to
+   // GetHeapFreeSpace(HEAP_ANY) for the true free heap.
+   size_t GetHeapFreeListSpace (void) const
+   {
+#if RASPPI >= 4
+       return s_pThis->m_HeapLow.GetFreeListSpace () + s_pThis-
>m_HeapHigh.GetFreeListSpace ();
```

```
+#else
+       return s_pThis->m_HeapLow.GetFreeListSpace ();
+#endif
+   }
+
    static void DumpStatus (void)

--- a/include/circle/pageallocator.h
+++ b/include/circle/pageallocator.h
@@ -48,6 +48,10 @@ public:
    size_t GetFreeSpace (void) const;

+   /// \return Onyx: bytes of freed pages currently on the free list (reusable). Add
+   ///     to GetFreeSpace() for the true total free space.
+   size_t GetFreeListSpace (void) const;
+
    void *Allocate (void);
@@ -67,6 +71,7 @@ private:
    TFreePage   *m_pFreeList;
+   unsigned     m_nFreeListCount;  // Onyx: pages currently on m_pFreeList
    CSpinLock    m_SpinLock;

--- a/lib/pageallocator.cpp
+++ b/lib/pageallocator.cpp
@@ -30,7 +30,8 @@ CPageAllocator::CPageAllocator (void)
-   m_pFreeList (0)
+   m_pFreeList (0),
+   m_nFreeListCount (0)
 {
 }
@@ -49,6 +50,11 @@ size_t CPageAllocator::GetFreeSpace (void) const
+size_t CPageAllocator::GetFreeListSpace (void) const   // Onyx: reusable freed pages
+{
+   return (size_t) m_nFreeListCount * PAGE_SIZE;
+}
@@ -68,6 +74,7 @@ void *CPageAllocator::Allocate (void)
        m_pFreeList = pFreePage->pNext;
+       m_nFreeListCount--;         // Onyx: page taken off the free list
@@ -103,6 +110,7 @@ void CPageAllocator::Free (void *pPage)
    m_pFreeList = pFreePage;
+   m_nFreeListCount++;             // Onyx: page returned to the free list
```

---

4. Multi-core enabled (`ARM_ALLOW_MULTI_CORE`)

`include/circle/sysconfig.h`: the define is un-commented, so Circle builds its multi-core support (`CMultiCoreSupport`, real spin locks, per-core stacks and MMU setup, IPIs). The Onyx kernel starts cores 1–3 at boot; core 1 is the **sound producer** (it renders the audio chunks ahead, see Kernel Internals §12); the scheduler, the processes and the interrupts stay on core 0. The memory map is unchanged (Circle always reserves the stacks of the 4 cores).

```
-//#define ARM_ALLOW_MULTI_CORE
+#define ARM_ALLOW_MULTI_CORE           // Onyx: core 1 runs the sound producer
(kernel sys/sound.cpp)
```

> Changing this define changes code in **every** Circle library: after updating the submodule, run `make clean` in each library before rebuilding them (the `.o` files do not depend on `sysconfig.h`) — see the Developer Guide §2.

5. Shift + navigation keys

Circle's keymap only has **Ctrl** variants of the navigation keys (`KeyCtrlUp` … → `ESC[1;5A` …); Shift + an arrow produced nothing. The patch adds eight special keys **at the end** of `TSpecialKey` (just before `KeyMaxCode`), so every existing code — the values stored in the `.kmap` layout files — keeps its number:

Key	String
`KeyShiftHome` / `KeyShiftEnd`	`ESC[1;2H` / `ESC[1;2F`
`KeyShiftPageUp` / `KeyShiftPageDown`	`ESC[5;2~` / `ESC[6;2~`
`KeyShiftUp` / `Down` / `Left` / `Right`	`ESC[1;2A` / `B` / `D` / `C`

The Onyx layouts (`tools/keymaps/maps/*.h` → `genkeymaps.py` → `SD:/etc/keymaps/*.kmap`) put them in the **Shift** column of the physical keys 0x4A–0x52. The kernel's key parser (`gui/window.cpp`, `NextKey`) turns any `ESC[n;mX` into the plain `KEY_*` code; apps read the modifier with `kapi_get_modifiers` (text selection in `uikit::Textarea` / `RichTextBox`).

6. Partial display update

`C2DGraphics::UpdateDisplay ()` sends the whole off-screen buffer to the frame buffer. The Onyx compositor redraws only the damaged rectangles of the screen, so the patch adds an overload that sends one rectangle:

```
void UpdateDisplay (unsigned nPosX, unsigned nPosY, unsigned nWidth, unsigned nHeight);
```

It clips the rectangle to the screen, copies its rows into one contiguous block (a screen-sized buffer allocated on first use: cached memory, fast) and hands it to `CBcmFrameBuffer::SetArea` — the frame buffer's own 2D DMA copy with the destination pitch, as the full update uses. With VSync (page flipping) or a display that is not the frame buffer it falls back to the full `UpdateDisplay ()`.

7. Yielding SD waits (FatFs lock + EMMC wait hooks)

The EMMC driver waits for the card in a busy loop (`CEMMCDevice::TimeoutWait`) unless `NO_BUSY_WAIT` is set, and Onyx's kernel is not preempted: every SD command kept the CPU, and a directory walk or a big read froze the GUI and the network 100–200 ms at a time. `NO_BUSY_WAIT` is global (USB, sound, every `CGenericLock`), so the patch adds two **weak** hooks instead; without a definition, Circle behaves exactly as upstream:

- `ffsystem.cpp`: `ff_mutex_take` / `ff_mutex_give` call `OnyxFsLockTake (vol)` / `OnyxFsLockGive (vol)` when they are defined, instead of the `CGenericLock` (a spin lock).
- `emmc.cpp`: once a `TimeoutWait` has spun 2 ms (the normal latency of a command; yielding earlier cost another task's whole time slice per command: reads fell to 2 MB/s), each turn of its loop calls `OnyxDriverWait ()` when it is defined.

The kernel defines them in `kernel/sys/fslock.cpp`: a sleeping, re-entrant volume lock (waiters `Yield`), and a wait hook that yields when IRQs are on. See `docs/02` (preemption).

7b (2026-10-01): many short waits. One command can now carry a whole file's contiguous clusters (patch 10): a long write waits for the card a few hundred µs per block, every wait under the 2 ms spin, and kept core 0 for 200 ms (`stall: jet:cache ran 213 ms without yielding`, the PC in `TimeoutWait`). `emmc.cpp` now also calls a second weak hook, `OnyxDriverPoll ()`, at each turn of a wait shorter than 2 ms. The kernel's (`fslock.cpp`) yields once the task has run 10

ms since its last `Yield` (the scheduler notes that time), with the same conditions as `OnyxDriverWait` (a scheduler on this core, IRQs on, not core 1's crash dump). Safe for the same reasons as patch 7: the volume lock is held (no other task sends the card a command — and nothing outside FatFs drives the card while tasks run), and the SD host holds a PIO transfer between two blocks until it is served (upstream's `NO_BUSY_WAIT` yields at every turn of these waits). Reads (40 MB/s) yield about every 400 KB, writes every 50–100 KB. Without the hook, Circle behaves as upstream.

8. Sector cache in the FatFs disk layer

FatFs reads the FAT and the directories one sector at a time (through its window), and each `disk_read` is a whole SD command round trip (plus a CMD13 status check in the EMMC driver): opening a file deep in a big folder, or listing it, asked the card again for every sector, every time. `diskio.cpp` now keeps the **single-sector reads** in a direct-mapped cache (2048 slots x 512 bytes = 1 MB, allocated on first use; key = sector and drive). **Write-through**: `disk_write` writes the card first, then updates the cached copies of the sectors written (or drops them if the write failed), so the cache never holds anything the card does not. The big multi-sector reads of file data go straight to the device. `disk_initialize` (a mount) and the device's removal forget the drive's sectors. `disk_cache_enable (0/1)` / `disk_cache_stats (&hits, &misses)` (declared in `diskio.h`); Onyx turns it off with `sdcache=0` in `cmdline.txt`.

Also: the bounce buffer for unaligned transfers is now **one per volume** (the SD driver can yield in the middle of a transfer since patch 7, so two volumes' transfers can overlap; FatFs locks per volume).

Test on the PC: `tools/tests/run_fs_test.sh` builds the fork's `ff.c` + `diskio.cpp` against stub Circle headers, formats a 96 MB RAM disk (FAT32) and runs 4000 random operations (create, overwrite, append, read back, delete, rename, list, remount) checked against a model, with the cache on and off: both pass and leave the same image byte for byte; the cache cuts the device reads about 3.4 times.

9. High Speed SD at run time

Upstream enables the SD High Speed mode (50 MHz, SD 1.1+ cards: CMD6 switch) only with the compile option `SD_HIGH_SPEED`, off by default, and then only raises the clock. The patch:

- `static void CEMMCDevice::SetHighSpeed (boolean)` (before `Initialize`) chooses it at run time (`SD_HIGH_SPEED` still forces the upstream behaviour); `IsHighSpeed ()` tells whether the card switched;
- before raising the clock, it sets the host's **High Speed Enable** bit (`CONTROL0` bit 2, `HCTL_HS_EN` of the SD Host Controller spec), so the controller samples with the High Speed timing;
- the 64-byte CMD6 status buffer is word aligned (the PIO transfer asserts it).

Onyx turns it on by default (`sdhs=0` in `cmdline.txt` turns it off, for a card that misbehaves).

10. Multi-cluster transfers in FatFs

`f_read` / `f_write` move whole sectors straight between the caller's buffer and the disk, but upstream **clips each transfer at the cluster boundary**: with small clusters (a card formatted with 512-byte clusters) every SD command carried 512 bytes, and the commands' latency (~240 µs each on the Pi 4) capped reads at 2 MB/s (`fsbench`: 1 MB = 2064 commands; the data port itself moved 1 MB in 23 ms). The patch goes on across the **next clusters while they follow on the disk** (`get_fat (cl) == cl + 1` for a read; `create_chain` — follow or allocate — for a write), up to the

size asked, then sets `fp->clust` to the cluster of the last sector moved, so the loop resumes exactly as before. A cluster allocated ahead that does not follow simply stays in the chain (the next turn finds it, as it would have allocated it). Not with FastSeek (`cltbl`). The allocation order is unchanged: `run_fs_test.sh` gets the **same disk image as upstream's `ff.c`**, byte for byte, with 14 x fewer reads and 6 x fewer writes on 512-byte clusters.

11. FatFs fast seek

`FF_USE_FASTSEEK` is **1**: `f_lseek` can use a file's **cluster map** (`FIL.cltbl`) instead of walking the FAT chain from the start at each backward seek. The kernel's `kapi_seek` (ABI v57) builds it at the first seek of a file bigger than 4 MB (`f_lseek (fp, CREATE_LINKMAP)`, the table grown once if the file is fragmented) and `kapi_close` frees it: a GameCube disc image (1.4 GB) is then read anywhere at once. The option adds `cltbl` to `FIL`: **the kernel must be rebuilt clean** with the new `libfatfs`.

12. 2D DMA with a source stride, asynchronous partial updates

Why. Circle's `SetupMemCopy2D` only knows a destination stride: the source must be one contiguous block. So patch 6 gathered each damaged rectangle's rows into a buffer with the CPU (a copy as big as the DMA's) before the frame buffer's DMA copied it to the screen, and `SetArea` then **busy-waited** for the DMA: core 0 (the GUI, the apps' tasks) stood still for the whole transfer (2–4 ms for a full 1080p screen).

What. Both DMA engines have a source stride too (legacy `STRIDE` bits 0–15, DMA4 `SRCI` bits 16–31):

```
void SetupMemCopy2D (void *pDestination, const void *pSource,
                     size_t nBlockLength, unsigned nBlockCount, size_t nBlockStride,
                     unsigned nBurstLength, size_t nSourceStride);  // CDMAChannel,
CDMA4Channel
void CBcmFrameBuffer::SetAreaPitch (const TArea &, const void *pPixels, unsigned
nSourcePitch,
                                    TAreaCompletionRoutine *pRoutine = nullptr, void
*pParam = nullptr);
void C2DGraphics::UpdateDisplayAsync (unsigned x, unsigned y, unsigned w, unsigned h,
// w = 0: all
                                      CDisplay::TAreaCompletionRoutine *pRoutine,
void *pParam);
```

The overloads clean only the rectangle's rows from the data cache (not the whole span). The partial `UpdateDisplay (x, y, w, h)` of patch 6 now uses `SetAreaPitch` (the gathering buffer is gone). `UpdateDisplayAsync` starts the DMA and returns; the routine runs from the DMA's interrupt.

Polled, without the interrupt. Under a heavy GPU load (Ocarina of Time in `n64emu`) a completion interrupt was now and then never seen, and the compositor waited for ever (the screen, the apps presenting, the sound feeder behind them: all stuck). So the compositor (`CCompositorTask::Present`, kernel.cpp) no longer uses `UpdateDisplayAsync`; it starts the DMA and polls it between yields:

```
boolean C2DGraphics::UpdateDisplayStart (unsigned x, unsigned y, unsigned w, unsigned
h); // FALSE: done at once
boolean C2DGraphics::UpdateDisplayPoll (void);                  // TRUE once the
transfer is over
boolean CBcmFrameBuffer::SetAreaPitchStart (const TArea &, const void *, unsigned
nSourcePitch);
boolean CBcmFrameBuffer::SetAreaPoll (void);                    // (frees the frame
```

```
buffer's DMA channel)
boolean CDMAChannel::Poll (void);  boolean CDMA4Channel::Poll (void);   // not active
any more
```

It draws nothing into the off-screen buffer until the poll says done. `cmdline.txt dispdma=0` still makes every copy synchronous (`UpdateDisplay`).

13. SD card partitions as volumes, exFAT

Why. Upstream maps each FatFs volume to a whole device (`emmc1`, `umsd1`…): only the first FAT partition of the card was seen, and exFAT was off (files ≤ 4 GB, no exFAT user partition). The Pi 4 boots only from a FAT32 partition 1; a second partition, formatted exFAT, can hold the big files (ROMs, disc images).

What.

- `ffconf.h`: `FF_VOLUMES 9`, `FF_VOLUME_STRS "SD","SD1","SD2","SD3","USB","USB2","USB3","FD","NVME"` (since §28: 21 volumes, the USB devices' `USB1`..`USB3` and their partitions `USB1P1`..`USB3P4`); `FF_FS_EXFAT 1` (needs `FF_USE_LFN`; `FSIZE_t` becomes 64-bit → rebuild the kernel clean).
- `ffconf.h`: `FF_MULTI_PARTITION 1`. `diskio.cpp`: the physical drives are `emmc1` (the whole card, as upstream), `umsd1`…`umsd3`, `ufd1`, `nvme1`; **`VolToPart`** maps `SD:` to the card's partition 0 (*auto*: FatFs takes the first FAT volume, exactly as before the change) and `SD1:`…`SD3:` to MBR partitions 2–4 of the same drive (their type is not checked: FAT or exFAT is recognised from the boot sector). `disk_initialize` keeps a drive already initialised (a second partition of it) instead of forgetting its sector cache. (A first version opened Circle's partition devices `emmc1-1`…: dropped, `SD:` must be found exactly as before.)
- `ff.c`: the multi-cluster **write** extension (patch 10) is skipped on exFAT: a contiguous exFAT file (`NoFatChain`) that becomes fragmented must first have its FAT chain written, which upstream does cluster by cluster (reads keep the fast path).

Rebuild everything that includes `ff.h` after changing `ffconf.h`: `FF_FS_EXFAT` makes `FSIZE_t` 64-bit, so `FIL` / `FATFS` / `FILINFO` grow. Besides `libfatfs` and the kernel (clean), that is **`addon/wlan`** (`libwlan.a`: the Wi-Fi firmware is read through FatFs) and **`addon/wlan/hostap/wpa_supplicant`** (`make -f Makefile.circle`: it reads `wpa_supplicant.conf`). With stale ones, their `FIL` on the stack is too small: FatFs writes past it on the network core — Onyx booted but the Wi-Fi never came up.

The kernel mounts `SD1:`…`SD3:` when `f_mount` succeeds and shares one lock slot between the four SD volumes (one card). `tools/tests/run_fs_test.sh` runs the FatFs test on FAT32 **and exFAT** images (the fork's image must equal upstream's), and first `fstest parts`: an MBR card image with partition 1 FAT32 and partition 2 exFAT, `SD:` and `SD1:` mounted, a file on each, `SD2:` absent.

14. Wi-Fi: a DHCP restart (join another network while running)

Why. `CWPASupplicant` reads `wpa_supplicant.conf` once, at boot, and once bound, Circle's DHCP client sleeps until its lease's renewal time: joined to another network, the Pi would keep the old address.

What. `CDHCPClient::Restart ()` (static flag `s_bRestart`, `include/circle/net/dhcpclient.h`, `lib/net/dhcpclient.cpp`): the bound state checks it every 0.5 s (it slept 10 s at a time) and, if set, halts the network and starts over (discover / request).

The configuration itself is read again **without touching hostap** (a submodule of upstream's): wpa_supplicant registers its SIGHUP handler (`wpa_supplicant_reconfig`: every interface reloads

its configuration, deauthenticates and joins the highest `priority` network in range) with `eloop_register_signal_reconfig`, which Circle's eloop ignores. The Onyx kernel is linked with `--wrap=eloop_register_signal_reconfig` (`kernel/Makefile`): it keeps that handler, and `kapi_wlan_reconnect` (ABI v60) runs it from the supplicant's own event loop (a 0 s eloop timeout), then asks `CDHCPClient::Restart ()`.

Not a patch: build configuration

One required setting is a **configure option**, not a source change: Onyx renders 32-bit pixels, so Circle must be configured with **`DEPTH=32`**:

```
cd circle && ./configure -r 4 -p aarch64-none-elf- -d DEPTH=32 -f
```

See the Developer Guide §2 for the full Circle build.

15. A crash area kept out of the heap

Why. A frozen Pi (a kernel hang, a bus wedged by the GPU) leaves nothing to read: `kmsg` is gone with the session. Onyx keeps a crash record in RAM that survives the watchdog's reboot.

What. `CMemorySystem::SetupHighMemAbove4G` (`lib/memory64.cpp`) keeps the top `ONYX_CRASH_AREA_SIZE` (64 KB) of the first RAM segment it adds above 3 GB (the `[3 GB, RAM top)` low-RAM top from the device tree, or the `[4 GB, RAM end)` chunk of the fallback — the case of an 8 GB Pi 4 without a captured device tree) out of the high heap and publishes its address in `u64 g_ulOnyxCrashArea` (`include/circle/memory.h`; 0 when the board has no RAM there, e.g. 2 GB).

`CSpinLock::TryAcquire ()` (`include/circle/spinlock.h`, `lib/spinlock.cpp`): one try, FALSE (nothing held, the IRQ level restored) when another core holds the lock — core 1's crash watch never waits for ever behind core 0. It stays mapped NORMAL (cacheable): the kernel cleans each write to the point of coherency. See docs/02 §13.

16. TCP: a peer gone before it was accepted

Why. A client that connects and closes at once (a port scan, a script checking a port) left the listening connection woken without a foreign address: `CTCPConnection::Accept` then called `CIPAddress::Set` on it, whose assertion (`rAddress.m_bValid`) is a panic — every core halted, the Pi restarted (seen with telnetd: `lastcrash.txt` showed it, in a restart loop while a monitor on the PC probed port 23).

What. `lib/net/tcpconnection.cpp`, `Accept`: after the wait, an error (`m_nErrno`) or a connection without a foreign address returns `-NET_ERROR_CONNECTION_RESET`; `CSocket::Accept` then returns 0 (that peer not accepted) and listens again. The same bug is in upstream Circle.

17. The host name set after the constructor

Why. The name DHCP announces is given to `CNetSubSystem`'s constructor, and Onyx builds it as a member of `CKernel` — before the SD card is mounted, so before `SD:/etc/system.ini` (its `hostname=`, written by Setup, the first-run wizard) can be read.

What. `include/circle/net/netsubsystem.h`: `void SetHostname (const char *pHostname)` (inline: `m_Hostname` replaced), called by the kernel before the network's bring-up task starts (`Initialize` makes the DHCP client with the name). Header only: no library to rebuild.

18. DNS: the answer polled, not a second slept

Why. `CDNSClient::Resolve` sent its query, slept 1000 ms, then read the answer once: every name lookup cost a second even when the answer came in 5 ms (a web page asks for several hosts: NetSurf spent seconds in DNS alone, its connects one after another).

What. `lib/net/dnsclient.cpp`: after each send, the socket is read every 5 ms (`MSG_DONTWAIT`) for up to 1000 ms; the three tries are kept. (The kernel also caches the answers: `kernel/sys/net.cpp`, docs/02.)

19. TCP: a 64 KB receive window

Why. The advertised receive window was 10 segments (14600 bytes), with no window scaling: a connection could not carry more than 14.6 KB a round trip (about 730 KB/s at 20 ms, 290 KB/s at 50 ms) -- well under the Wi-Fi's rate.

What. `lib/net/tcpconnection.cpp`: `TCP_CONFIG_WINDOW` is 44 segments (64240 bytes, the most a 16-bit window field holds); the reassembly queue follows it. Rebuild `lib/net`.

20. TCP: real duplicate ACKs only

Why. `CTCPConnection::PacketReceived` counted **every** segment whose ACK did not advance as a duplicate ACK — the peer's own data segments too, and ACKs with nothing in flight, window updates — and the count was reset only by an advancing ACK. The remote desktop client (Onyx Remote) sends small messages (READY, the pointer's moves) while the Pi streams the screen: three in a row between two ACKs started a **spurious fast retransmit + fast recovery** (cwnd cut to about two segments), and the Pi's sending crawled while the mouse moved.

What. `lib/net/tcpconnection.cpp`: in the "ACK not advancing" branch, `OnDuplicateAck` is called only for a duplicate ACK as RFC 5681 §2 defines it: no data and no SYN / FIN (`nSEG_LEN == 0`), the ACK number SND.UNA, the advertised window unchanged (`m_nSND_WND`), and data outstanding (`FLIGHT_SIZE > 0`). Other segments neither count nor reset the count. Fast retransmit and fast recovery (NewReno) are unchanged otherwise. The same bug is in upstream.

21. TCP: the retransmission timeout

Why. `retranstimeoutcalc.cpp` started at 3 s (RFC 1122's value) and never went below 1 s (RFC 6298's conservative minimum): on a LAN or Wi-Fi peer a few ms away, a segment lost at the end of a burst (nothing behind it to cause duplicate ACKs) stalled the stream a whole second — the remote desktop froze.

What.

- `INITIAL_RTO` 1 s (RFC 6298 2.1); `MIN_RTO` stays **1 s**. A 200 ms minimum (Linux's `TCP_RTO_MIN`) was tried on 2026-10-01 and undone the same day: the remote desktop's throughput over Wi-Fi to a Windows PC collapsed (64 KB sends taking 0.5–2 s instead of a few ms, ~50x slower). Windows delays its ACKs up to 200 ms and Wi-Fi adds bursts of delay: the timer fired spuriously and each timeout cut the congestion window to one segment; Linux survives that floor with F-RTO / undo of spurious timeouts, which Circle lacks. RFC 6298's granularity term G is 200 ms (was 1 tick).
- **Karn's algorithm** also after a fast retransmit: `CRetransmissionTimeoutCalculator::SegmentResent` (new) marks a segment sent again by `ResendSegment` (fast retransmit, a partial ACK in fast recovery), so its ACK gives no RTT sample (only a timeout marked it before). A valid sample resets the backed-off RTO, as before.

- `MAX_RETRANSMISSIONS` stays 5 (63 s from 1 s), `MAX_SYN_RETRANSMISSIONS` 5 (63 s from the 1 s initial RTO; it was 189 s from 3 s).
- After a SYN retransmitted, RFC 6298 (5.7) wants the data phase to start at 3 s: the RTO is then the backed-off one (≥ 2 s) until the first sample — close enough, left as is.

22. `CSocket::Send`: the bytes queued

Why. `CSocket::Send` cuts a buffer into MSS-sized chunks and queues them one by one; when the connection's queue stays full past the send timeout (Onyx sets 5 s), a chunk fails — and `Send` answered the **error**, although the chunks before it were queued and would be sent. The kernel's `NetTcpSend` (32 KB requests) then counted none of that request, and an app sending "the rest" again put those bytes **twice** in the stream (the remote desktop's stream broke: its client read garbage).

What. `lib/net/socket.cpp`: `Send` (and `SendTo`, for TCP) answers the bytes queued when a later chunk fails, if any (as POSIX `send`); the error comes back at the next call. The kernel's `NetTcpSend` adds its requests' counts and stops at a short one.

Test (patches 20–22): `tools/tests/run_circlenet_test.sh` builds the fork's real `tcpconnection.cpp`, `retranstimeoutcalc.cpp`, `socket.cpp` (+ the buffers, queues, checksum) on the PC against stub Circle headers (`tools/tests/circlenet/stub`: a simulated clock and kernel timers, no tasks, a network layer that hands each segment to the test) and plays the peer: the peer's data segments and window updates are no duplicate ACKs, three real ones still start a fast retransmit and a full ACK ends the recovery, the RTO (200 ms, backed off, reset by a sample, 1 s for a SYN, Karn after a fast retransmit, a dead peer given up after 51 s), and `Send`'s count after a timeout. Against the unpatched sources it fails 15 of its 25 checks.

23. Sockets for the BSD layer: `AcceptReady`, a connection kept until its socket lets it go

Why. The kapi v75 BSD sockets (docs/02 §8, `sys/bsdsock.cpp`) need a non-blocking accept and sockets that stay valid after a reset until `close`. Upstream deleted a terminated connection at the next `Process` even while a `CSocket` still held its handle; after a reset the handle went to the next connection, which the old socket then read, wrote and closed.

What (fork commit `f1d6b200`; `tools/circle-patches/wp-net.patch`): `CSocket::AcceptReady ()` (a backlog connection already connected, so `Accept` will not block; it also replaces backlog connections that died before being accepted); `CNetConnection::SetReleased` / `IsReleased`, set by `CTransportLayer::Disconnect` and by a failed `Connect` — a terminated connection is deleted only once released; `CSocket::Accept` disconnects a failed backlog connection instead of leaking it; `CTransportLayer::IsTerminated (h)`.

Test: `tools/tests/run_circlenet_test.sh` (the stub gained `IsTerminated`): 25/25.

24. `CPageAllocator::Allocate`: a failed attempt leaves the count as it was

Why. `Allocate` advanced its bump pointer (`m_pNext += PAGE_SIZE`) before checking it against `m_pLimit`, and returned 0 without stepping it back: every failed attempt left the pointer one page further past the limit, and `GetFreeSpace ()` (`m_pLimit - m_pNext`, a `size_t`) wrapped. `palloc_high` tries the high segments in order, so once segment 0 was full every page served elsewhere cost it one more page: `meminfo`'s free memory fell for ever (an OOM kill seemed to lose ~1.1 GB on an 8 GB Pi) and the kernel's OOM check fired with half the memory still free. No frame was lost.

What. `lib/pageallocator.cpp`: `m_pNext -= PAGE_SIZE` before that `return 0`.

Test: `memtest oom` on the Pi (the OOM kill twice, each run ending where it started).

25. TCP: a closed connection's retransmission timer

Why. The Pi restarted by itself under network load (a video streamed by the WebKit browser, an FTP upload of 100 MB and the remote desktop at once; a telnet client gone while output flowed was enough): `SD:/etc/lastcrash.txt` said **`A kernel panic: tcp: Unexpected state 0 at line 1922`**. That line is `CTCPConnection::TimerHandler`'s retransmission case: the timer of a connection whose state is `Closed`. A peer's RST, the ACK of our FIN in `LAST-ACK` or a refused connect closed the connection with the retransmission timer still running; since §23 a terminated connection is kept until its socket releases it (upstream deleted it at the next `Process`), so the timer fired on the closed connection a second later. Upstream calls that `UNEXPECTED_STATE ()`, which in a build without `NDEBUG` — ours — is `LogPanic`: every core halted, for one connection, and any client could cause it.

What. `lib/net/tcpconnection.cpp`:

- every path of `PacketReceived` that goes to `Closed` (a RST in each state, a SYN in the window, the FIN's acknowledgement in `LAST-ACK`, a refused connect) stops the retransmission timer first;
- `TimerHandler`, `TCPTimerRetransmission`: in `Closed`, `Listen`, `FinWait2` or `TimeWait` the timer is ignored before the RTO is backed off or the retry count taken (the handler runs from an interrupt, with `netcore=1` on another core than the stack: it can still race those paths);
- a reset (and the SYN case) also sets `m_TxEvent`: a sender waiting for room in the transmit queue sees the reset at once instead of at its timeout.

Test: on the Pi — YouTube playing in Web with an FTP upload of 100 MB and telnet sessions opened and dropped: no panic since (it came within minutes before).

26. Wi-Fi: the chip polled, a scan over both bands, 5 GHz preferred

Three changes in the BCM4343x driver (`addon/wlan/ether4330.c`), each behind a global the kernel sets (`kernel/sys/net.cpp`: `NetWlanOptions`, `NetWlanNames`); with the globals at their defaults the polling and the bias are off (the scan over both bands is not conditional). `hostap` — upstream's submodule, wpa_supplicant and its Circle driver glue — is **not** changed.

1. The chip polled (`onyx_wlpoll`). The receive loop (`rproc`) read frames until the chip had none, then slept in `intwait` until the SDIO card interrupt. That interrupt comes late: measured on a Pi 4 under a steady stream, **2.5 to 5 ms a wait, 80 % of the reader's time**, with the frames already there — the Pi sent 1.1 MB/s and received 1 MB/s on a link good for forty times that. With `onyx_wlpoll` set (the kernel does, when the network has a core of its own: `netcore=1`), the reader asks the chip itself, in turn with the stack's other tasks — a round of that core's scheduler is ~13 µs: `intpoll` reads the chip's interrupt status (function 0's `Intpend`, then the backplane's `Intstatus`, acknowledged) and tells whether `FrameInt` is pending; the data function is read only then, and until it is empty (as Linux's brcmfmac: no read with nothing pending). On the primary core (`netcore=0`) the driver waits for the interrupt as before: polling would take the core from everything else.

2. A scan over both bands, the networks probed by name. `wlscanstart` listed the fourteen 2.4 GHz channels with the wildcard SSID alone, so a 5 GHz BSS was never a candidate: a Pi 4 beside a dual-band access point joined it on 2.4 GHz channel 1, shared with the neighbours. The escan now passes `nchans = 0` (the firmware's own list: every channel of both bands the country allows) and, beside the wildcard, up to four names (`onyx_scan_ssid`, `onyx_scan_nssid`): an access point

that leaves its name out of the beacons of one band (seen on a Livebox: the 5 GHz BSS came with an empty SSID) answers a probe by name and is seen as that network. The kernel fills the names from the `ssid="…"` lines of `SD:/etc/wpa_supplicant.conf` before wpa_supplicant starts and at each `wlan_reconnect`. A scan takes longer (5 GHz's DFS channels are listened to, not probed): `wlan_scan` waits 5 s instead of 3.5.

3. 5 GHz preferred (`onyx_scan_5g_bias`). wpa_supplicant ranks the BSSs of a network by level, and 2.4 GHz is usually heard a few dB louder. `wlscanresult` adds the bias (the kernel: 25 dB) to the level of each 5 GHz BSS heard at −78 dBm or better before the result is queued (kept negative), so a network on both bands is joined on 5 GHz unless that band is much weaker. The kernel's own scan (`wlan_scan`, `wifiscan`) clears the bias for its length: its list shows the levels as heard. (The glue reads the level unsigned — `res->level = bss->RSSI`, a `u16`: −67 is 65469 — an upstream bug left as it is: the order stays the same.) `WLC_SET_ASSOC_PREFER` was tried first and changes nothing here: the supplicant names the BSSID it joins.

`addon/wlan/p9util.cpp` gains `p9usec` (the microsecond clock) and `p9yield` (a turn of the scheduler) for the driver's C code. With `onyx_wlstat` (the kernel: `cmdline.txt netstat=1`) the driver logs every 5 s under load its frames a second, a frame's read and write times, the waits, and every 10 s what the firmware says of the link (rate, RSSI, chanspec, power save).

4. The frames' path (`onyx_wlfast`, a bit each; the kernel sets 31). With the chip polled and 5 GHz joined, a frame's read still took ~270 µs — 3700 frames a second at best:

- *bit 1 — one command a transfer* (`packetrw`): `sdiorwext` sent a length over a block (512) as its whole blocks in one CMD53 and the rest in a second, byte-mode one: a 1500-byte frame was the header (12 bytes), two blocks, then 464 bytes — three commands. A length over a block is now rounded up to whole blocks (the "roundup" of Linux's brcmfmac: past a frame's end the chip pads a read and ignores what is written).
- *bit 2 — the next frame read whole* (`wlreadpkt`): a frame's header tells the next frame's length (`nextlen`, 16-byte units) when the chip has one queued; that frame is then read in one command instead of its header first. A hint too short is completed by a second read.
- *bit 4 — the frames' locks without a yield* (`fqlock` / `fqunlock`, `emmcio`): `qlock` and `qunlock` each call the scheduler, free or not, and `tsleep` yields before it looks at its condition — a frame cost eleven turns of the scheduler (the packet lock, the SDIO lock around each command, the wait for each transfer's end), each a round of all the stack's tasks. On the frames' path a free lock is taken and released at once, a transfer already ended is not waited for, and the reader yields once a frame. (Doing this in `p9proc.cpp` for every lock and sleep of the driver cut the Wi-Fi at boot — not found why; only this path is changed.)
- *bit 8 — the bus at 50 MHz* (`emmc.c`): the driver switches the card to High Speed (function 0's register 0x13) but the host stayed at 25 MHz; when the bus goes to four lines the host now takes High Speed timing and 50 MHz, as Linux runs this chip on a Pi 4. (The controller's base clock is 250 MHz there: the divider gives 41.7 MHz. The next step, 62.5 MHz, was tried: the chip does not answer.)
- *bit 16 — the controller's registers written without a wait* (`emmc.c`, `WR`): each write waited 2 µs first — two periods of the SD clock, which is 2 µs at 1 MHz and 40 ns at 50 —, and a command writes eight registers.

A frame's read: **93 µs** (was 270), a frame's write 16 µs (was 36).

Receive glomming was tried and taken out. `bus:rxglom` is accepted by the firmware, which then expects the glom header on what it is *sent* too (brcmfmac's `txglom`: 8 bytes between the length words and the software header — without it the chip stops answering commands); with both done the Wi-Fi works, but this firmware (the 43455's) never sent a superframe in 50 MB received, and the rates were a little lower.

Measured (a Pi 4, `tcpbench` against a PC on the same access point; with the kernel's inter-core interrupt of docs/02 §11, which came first, and §27's TCP changes, which came last):

	echo round trip	the Pi sends	the Pi receives
before	10.4 ms	795 KB/s	504 KB/s
the IPI (kernel)	2.8 ms	1128 KB/s	1008 KB/s
+ the chip polled	2.8 ms	3.3 MB/s	1 MB/s
+ 5 GHz (VHT, 80 MHz, link 195–292 Mbit/s)	2.2 ms	4.3 MB/s	4.4 MB/s
+ TCP window scaling (§27)	2.3 ms	6.6 MB/s	4.6 MB/s
+ one command a frame, the locks	2.3 ms	6.4 MB/s	5.5 MB/s
+ the bus at 50 MHz	2.2 ms	7.5–8.1 MB/s	6–7 MB/s
+ the registers written without a wait	2.1 ms	8–9.5 MB/s	**8.7 MB/s**

From the internet (the Pi's `curl`): 30 MB from Cloudflare at 7.1–8.5 MB/s (it was 0.8, then 2.2–2.8 with the first four rows); YouTube's 3 MB player script in 0.6 s (was 4 to 23 s). No error of the driver in 4 × 64 MB each way.

What limits it now (`netstat=1`): 6000 frames a second at 93 µs each is over half the reader's time — 61 µs of it is the frame's data on the bus —, and the Pi acknowledges every segment (6000 frames sent a second while it receives). What is left: delayed acknowledgements. The bus is on four lines (`Busifc` 2) and the link is VHT (`vhtmode 1`, chanspec `e02a`): neither is the limit.

5. The frames sent are not aggregated (`onyx_wl_ampdu_tx = 0`: the firmware's `ampdu_tx`). The symptom was the user's: *while the browser loads a page, the remote desktop freezes*. Measured: during a download from the internet the PC's pings of the Pi lost 20 % of their answers, and **50 to 75 % during a download limited to 1 MB/s** (none when idle); an echo on another TCP connection came back after 1, 3, 7 seconds (the Pi's retransmission timeouts: its minimum is 1 s), a telnet session stalled for a minute. Where the packets went, step by step:

- the Pi's network core is never held (its longest scheduler round in 5 s: 0.65 ms);
- the Pi **receives every request and writes every answer to the chip** (the driver's counts of ICMP frames handed up and taken down: 150 / 150), never keeping a frame more than 19 ms;
- the PC receives a third of them (Windows' counters: no damaged packet, just fewer), and the Pi's own pings to the gateway are lost alike at that time (8 answered of 25): it is **what the Pi sends** that disappears, whatever its destination — its TCP acknowledgements too, which a download does not notice (they are cumulative);
- the firmware counts them as sent and acknowledged (its `counters`: `txfail` 2 in 3453 frames).

So the frames are lost after the access point's block acknowledgement — in the firmware's or the access point's handling of the aggregates (an Orange Livebox, 5 GHz, VHT 80 MHz, −70 dBm). What changes it: **`ampdu_tx` 0 — none lost** (`wlinit`: the interface taken down, the setting, up again;

it is refused while up). What does not: a 40 MHz channel (`bw_cap`), a block acknowledgement window of 8 (`ampdu_ba_wsize`), no RTS (`ampdu_rts`), no A-MSDU, a receive window of 8. The price is the Pi's own sending rate — each frame is a transmission of its own — and the acknowledgements of what it receives: hence §27's delayed acknowledgements.

the kernel's defaults now	the Pi receives	the Pi sends	from the internet	the PC's pings lost during a download	an echo during a download
A-MPDU on what is sent (the table above's last row)	8.7 MB/s	8–9.5 MB/s	7–9 MB/s	20–75 %	up to 7 s
no A-MPDU on what is sent, an acknowledgement for 8 segments	3.8–4.8 MB/s	1.3–1.7 MB/s	5.6–5.9 MB/s	**0 %**	**77 ms at worst**
+ frame bursting (`onyx_wl_frameburst = 1`: the firmware's command 219, which Linux's brcmfmac sets too — several frames in one transmit opportunity)	4 MB/s	**3.1 MB/s**	6.5 MB/s	**0 %**	0.3 s at worst

The settings are the driver's globals (`onyx_wl_ampdu_tx`, `_ampdu_rx`, `_ba_wsize`, `_ampdu_rts`, `_rx_ba_wsize`, `_bw5`, `_frameburst`; −1: the firmware's own), each a word of the trial file. **To do**: whether a newer firmware aggregates soundly (the sending rate would be 6 to 9 MB/s again).

Trying such a change on a Pi that is only reachable by its Wi-Fi: the kernel's one-boot trial file (docs/02 §11 *A trial*): each of the bits above was tried alone that way before it became the default.

The driver tells when the chip was last busy

`ether4330.c` keeps two figures for the kernel's network core, which sleeps between two questions to the chip once the network is quiet (docs/02, *The network core sleeps when the network is quiet*): `onyx_wl_lastact`, the clock when the chip last said it had something, when a

frame was last read from it or written to it; `onyx_wl_polls`, the times the polling reader asked the chip and it had nothing. No change of behaviour in the driver itself.

`emmc.c`: **`sdiocardintrpending (arm)`** tells whether the card's interrupt is pending, from the controller's flag (no command) and, when it is not and `arm` is set, enables that interrupt — its handler then sends an event (`sev`) that ends the network core's sleep. Two things had to be put right for it. The flag is the line's level only while its status is enabled: written alone it stays set after the data lines moved during the last commands (it read "pending" for ever) — its status is disabled, then enabled again, and it is sampled anew, as Linux's SDHCI driver does. And the interrupt enable register is changed from two cores, the driver's tasks setting bits and the handler clearing those that came: every such change is now made under one lock (`irpenable`, `mmcinterrupt`). `sdiodebugreg` gives the controller's registers to the kernel's statistics line.

27. TCP: window scaling, a receive window that follows the queue

Why. The receive window was a constant — 64240 bytes (§19), never scaled, never smaller: (1) a connection carried at most 64 KB a round trip — 2.5 MB/s at 25 ms from the internet, and 4.5 MB/s on the LAN, where the queueing makes the round trip 14 ms; the Pi's own sends were held to 64 KB too (the peer's window read without its scale, a 64 KB transmit threshold); (2) a reader that stopped reading did not stop the peer: the receive queue grew without a limit (a paused download, a video's loader that waits).

What. `lib/net/tcpconnection.cpp` (the switch `onyx_tcp_ws`, set by the kernel; 0: as before):

- **the option**: our SYN carries window scale 3 (after the MSS: NOP, kind 3, length 3); a SYN+ACK carries it when the peer's SYN did. `ScanOptions` takes the peer's shift (in a SYN, in `SYN-SENT` or `LISTEN`); with both, the peer's window field is shifted left by its shift (never in a SYN), ours right by 3, and the receive limit is 180 segments (262800 bytes). `TCP_MSS_HEADER_LEN` is 28 (the buffer's headroom).
- **the window** sent is the room left in the receive queue (`ReceiveWindow`: the limit less the bytes queued), in every segment; `Receive` — the reader made room — sends a window update once that opens the window by a quarter of the limit over what the peer was last told.
- the transmit threshold (when `Send` waits, when a socket is writable) is 256 KB, the initial slow-start threshold 1 MB with a scaled peer (it was 65535: linear growth past 64 KB).
- `ScanOptions` stops at an option of length under 2 (it looped for ever).
- **delayed acknowledgements** (`onyx_tcp_ws` bit 2, `onyx_tcp_ackn`: the kernel sets 8): every data segment was acknowledged by a segment of its own — 6000 frames a second sent while receiving 9 MB/s, each a transmission on the radio once they are no longer aggregated (§26, 5). A full segment (1000 bytes or more) in order, without PUSH or FIN, that fills no hole, is counted; the acknowledgement goes with the eighth, or from `Process` 10 to 20 ms after the first (`m_nAckPending`, `m_nAckPendingTicks`); a short segment (a request, a keystroke), a PUSH, an out-of-order segment are acknowledged at once, as before. Any segment sent with ACK clears the count.
- (`onyx_tcp_trace`, the trial's `netstat=1`: a line at each retransmission timeout — the connection, its sequence state, how long ago the peer's last segment came.)

Measured: the table of §26 (the Pi sends 4.3 → 6.6 MB/s by this alone); the download from Cloudflare 2.2–2.8 → 6.0 MB/s with §26's fast path.

28. USB volumes: format, labels, an unmount while a call waits, the eject

Why. kapi v93 (docs/02 §18) mounts the USB mass-storage devices as `USB1:`, `USB2:`, `USB3:` (or `USB1P1:`... per partition) when they are plugged in and unmounts them when they are ejected or pulled out, formats a volume and shows its label. Upstream's FatFs configuration has neither `f_mkfs` nor the labels, and FatFs assumes a volume is not unmounted while a call on it waits for its lock.

What.

- `ffconf.h`: **`FF_USE_MKFS 1`**, **`FF_USE_LABEL 1`** (`f_mkfs`, `f_getlabel`, `f_setlabel`). Neither changes a structure: no clean rebuild, `libwlan` / `wpa_supplicant` unaffected (unlike §13).
- `ff.c`: an unmount while another task waits for the volume lock (a USB stick ejected or pulled out: the kernel unmounts it holding the lock, the waiter goes on once it is given). **`validate`** tests the object again once the lock is held (`fs_type`, `id`; before, only the drive's status): a write on a volume unmounted meanwhile was carried out on the stale `FATFS` — on an ejected stick still plugged in, written to it behind the user's back. **`mount_volume`** checks `FatFs[vol]` is still the object it locked: it was re-mounting the unregistered object (an open after the eject found the volume back by itself). Both now return `FR_INVALID_OBJECT` / `FR_NOT_ENABLED` and give the lock back.
- `ffsystem.cpp`: with the kernel's lock hooks (§7) the `CGenericLock` is never used; `ff_mutex_delete` (at an unmount) no longer deletes it and `ff_mutex_take` / `_give` call the hooks before asserting it exists: a task that took the lock before the unmount gives it back after, which asserted (a kernel panic) before. `ff_mutex_create` keeps the existing one (a remount). Without the hooks: upstream's behaviour.
- **The USB devices' partitions** (the user's naming, 2026-10-06): `ffconf.h` **`FF_VOLUMES 21`**, the strings `"SD","SD1","SD2","SD3", "USB1","USB1P1".."USB1P4", "USB2",.., "USB3",.."USB3P4", "FD","NVME"`; `diskio.cpp`'s **`VolToPart`**: `USBn` = `{n, 0}` (the device whole: its first FAT volume, a superfloppy or its only partition), `USBnPm` = `{n, m}` (MBR partition m). The kernel mounts one or the other by the device's sector 0 (docs/02 §18). `ff.c`: its check **`FF_VOLUMES <= 10` lifted to 32** — it only guards the numeric `0:`..`9:` form (`get_ldnumber` reads one digit), which Onyx does not use; the volumes are reached by name. The structures do not change (no clean rebuild; `tools/tests/run_fs_test.sh` lifts the same check in its copy of upstream's `ff.c`).
- `diskio.cpp`: after each transfer with a drive other than the SD card (`pdrv != 0`), the weak hook **`OnyxDriverPoll`** (the kernel's, §7b: it yields once the task has run 10 ms). The USB driver waits in a busy loop (`NO_BUSY_WAIT` off): a format, a big folder kept core 0 for seconds. Between two transfers is a safe point: the drive's lock is held (the kernel's `fslock.cpp`: one lock per physical drive, so one for all the volumes of a USB device), it is called once the bounce buffer and the sector cache are done with, and a device removed meanwhile only fails the next transfer (`disk_removed` cleared `s_pVolume`).
- `usbmassdevice.{h,cpp}`: **`IOCtl (DEVICE_IOCTL_SYNC)`** — FatFs' `CTRL_SYNC` — sends **SCSI SYNCHRONIZE CACHE (10)** (whole medium); a device that refuses it (no cache) is reset to a known state and the call succeeds. The kernel sends it at an eject, after a format and at the session's end.

Tested on the PC: `tools/tests/run_fs_test.sh` builds `tools/tests/fs/usbtest.cpp` with the fork's FatFs and `FF_FS_REENTRANT 1` (a counting lock of its own): sticks formatted as FAT32 / exFAT / FAT16 with an MBR (as Windows), a FAT32 superfloppy, all found by `USB1:`'s auto search; a device of two partitions (`USB1P1:` FAT32, `USB1P2:` exFAT) then made one again by a format of `USB1:`, labels set and read; a stick pulled out while a file is written (errors, the lock free, the old objects invalid on the next stick); an unmount while a call waits for the lock — which fails

with upstream's `ff.c` (the write accepted, the open succeeding). Not tested on the Pi yet (docs/HANDOFF.md).

29. The Pi 5's high RAM registered once

Why. On the Pi 5 (`RASPPI = 5`) `MEM_HIGHMEM_END` is 8 GB and the RAM is one block (the I/O is above 64 GB, no hole at 3-4 GB): `SetupHighMem` already makes seg0 = [1 GB, min (RAM, 8 GB)). Onyx's `SetupHighMemAbove4G` (§15 and the high-memory patch) then found nothing above 8 GB in the device tree and its fallback (B) registered [4 GB, 8 GB) **a second time**: two page allocators handing out the same frames on an 8 GB board.

What. `lib/memory64.cpp`, under `#if RASPPI >= 5`: `SetupHighMem` takes the crash record's 64 KB (`ONYX_CRASH_AREA_SIZE`, `g_ulOnyxCrashArea`) from seg0's top, and `SetupHighMemAbove4G` is not called. RAM above 8 GB (a 16 GB board) stays unused until the Pi 5's user window moves above it (docs/PI5-PORT.md §5.2, policy B). `g_ulOnyxCrashArea` is defined before `SetupHighMem` now. The Pi 4's code is unchanged (its object differs only by the `assert` line numbers). Fork commit `065871f9`; built in `circle5/` by `tools/pi5/circle5.sh`. **Not tested on a Pi 5 yet.**

30. The net device chosen at run time

Why. `CNetSubSystem` is made once, in `CKernel`'s constructor, with its device type (Onyx: the WLAN). The Pi 5 has Gigabit Ethernet (the RP1's MACB, which Circle drives): Onyx takes the cable when its link comes up at boot, else the Wi-Fi (`system.ini` `network = auto | ethernet | wlan`, docs/04) -- a choice made after the stack is initialized, since the MACB is brought up by `CNetSubSystem::Initialize`.

What. `CNetDeviceLayer::SetDeviceType (TNetDeviceType)` (`include/circle/net/netdevlayer.h`, `lib/net/netdevlayer.cpp`): the type set and the device forgotten, so the next `Process ()` looks it up again. Called before the link is up (`CNetSubSystem::Process` starts the DHCP client on the first device running), so no address was asked on the other one. The kernel (`kernel/kernel.cpp`, `CNetBringupTask`) calls it through `GetNetDeviceLayer ()`. Unused with `network = wlan` (the Pi 4's default): the Pi 4 runs as before. **Not tested on a board yet.** Fork commit `ec821a94`.

31. The Pi 5's presented pixels made opaque

Why. The first test on a board (a Pi 500, 2026-10-09, a tester's report): the boot log showed, then the screen went black when the desktop came, the ACT LED still blinking (the kernel alive). Onyx's pixels are `0x00RRGGBB` (the alpha byte 0) and the Pi 5's display takes the alpha: the log's text, Circle's colours with alpha 255, showed; the desktop, alpha 0, was transparent. `framebuffer_ignore_alpha=1` in `config.txt` (on the Pi 5's card) was not honoured there.

What. `C2DGraphics::Opaque` (`include/circle/2dgraphics.h`, `lib/2dgraphics.cpp`, `#if RASPPI >= 5 && DEPTH == 32`): the rectangle's alpha bytes set to 0xFF in the off-screen buffer before it is sent (`UpdateDisplay`, the partial `UpdateDisplay`, `UpdateDisplayAsync`, `UpdateDisplayStart`) -- one pass over the presented rows, before the DMA cleans them from the cache. The Pi 4 is unchanged. With it, the kernel's direct mode (`fullscreen_direct`: a program writing the displayed framebuffer itself, n64emu and gcemu) is off on the Pi 5 (`MapScreen` gives 0: the programs present through `fullscreen_begin` / `present_fb`). Fork commit `9266885b`. **Switchable** (2026-10-10): `C2DGraphics::s_bOnyxOpaque` (default TRUE), set from the kernel's `cmdline.txt` `opaque=` -- the tester found that the Pi 500's latest bootloader honours `framebuffer_ignore_alpha=1` (Raspberry Pi OS also works around it in software); `opaque=0` skips the pass and gives the direct mode back. The Display applet has the check box. Fork commit `b47451c6`. **Tested on a Pi 500 (2026-10-10): the desktop shows.**

32. The Japanese keyboards' Ro and Yen keys

Why. A Japanese (JIS) keyboard -- the Raspberry Pi 500's and keyboard's `JP` variant -- has two keys past the layout table's 128 rows: Ro (`\ _`, USB usage 0x87, International1) and Yen (`¥ |`, 0x89, International3); `CKeyMap::Translate` dropped them.

What. `lib/input/keymap.cpp`, `CKeyMap::Translate`: 0x87 is taken at row 0x7D and 0x89 at 0x7E (Paste and Find, which no layout maps), where `SD:/etc/keymaps/JP.kmap` has them. The raw keys (`kapi_key_held`, the games) are unchanged. Not the keymap decoupling of §1 (its skill re-applies that one after an upstream merge: check this fold is still there too). Fork commit `9a9180ba`. **Not tested on a Japanese keyboard yet.**

Contributions to upstream Circle

The fork's changes useful to every Circle user are prepared as clean pull-request branches on upstream `develop` (the scheduler's `WakeTasks` race, the SD High Speed fixes, large heap blocks reuse, partial display updates with a DMA source stride, the DHCP restart), with the patches and the pull-request texts: see `docs/circle-upstream/README.md`.

Upstream merges

2026-10-09: Circle 51.1 and 51.1.1 (84 upstream commits after `Step51`, merge `c06e5a52`). What Onyx gains: the xHCI endpoint recovered from Halted after a transfer error (#704: the Pi 4's and the Pi 5's USB); transient HID report errors tolerated, a generic HID mouse-shaped report taken as a mouse (#693); the 8BitDo controllers, keyboards and Wireless Adapter 2, the Xbox 360 wireless receiver (and clones); Wi-Fi: the eero 7 routers (#696) and the hidden networks (hostap at `circle-step51.1`); FatFs R0.16 patch 2 (the Onyx patches merged automatically: `tools/tests/run_fs_test.sh` passes); the lost wake-up of `CSynchronizationEvent` (`BlockTask` checks the state under its lock) — **ported into Onyx's own scheduler** (`kernel/sched/scheduler.cpp`, `kernel/compat/circle/sched/scheduler.h`: it replaces Circle's); for the Pi 5: the official DSI touchscreens (`addon/rp1dsi`), the RP1's I2C buses 4 and 6. Upstream's DWHCI / `usbboost` / USB-MIDI work is for the Pi 1-3 only. Three conflicts: `usbkeyboard.h` (our `GetKeyMap ()` beside upstream's protected `ConfigureKeyboard`), `ether4330.c` (our scan kept — both bands, the configured networks probed by name — with upstream's directed probe for a hidden network added as one more name), `tcpconnection.cpp` (our duplicate-ACK test kept: patch 20, the same RFC 5681 test without resetting the count on another segment). The keymap decoupling (patch 1) came through intact. Tested on the PC (`run_fs_test.sh`, `run_circlenet_test.sh`, `run_ramfs_test.sh`, `run_gui_test.sh`, `run_gamepad_test.sh`). **Tested on the Pi 4 (the user, 2026-10-09):** the Wi-Fi joins, TCP holds (YouTube audio and video in Jet, a remote desktop session over rdpd), the Game Boy Color emulator plays with its sound. Not checked yet: a keyboard unplugged and plugged again (its layout kept), the 8BitDo / Xbox 360 wireless devices.

Updating the fork (submodule)

```
# merge a newer upstream into the Onyx branch, inside the submodule (a merge, not a rebase: the
# branch is published and the superproject pins its commits):
git -C circle fetch upstream --tags
git -C circle merge upstream/master            # resolve conflicts in the patched files
# then the skill resync-circle-keymap-patch (the keymap decoupling), and the Pi 5's tree: sh tools/pi5/circle5.sh
# rebuild the affected libraries: libcircle (heap + page accounting), libinput (keymap)
```

```
# then record the new fork commit in the superproject:
git add circle && git commit -m "Bump circle submodule"
```